

EKU Department of Computer Science

# Tiered Coalition Formation Games Empirical Study

CSC 495, Spring 2025

Seth Kietz  
Under Supervision of Dr. Nathan Arnold  
May 7<sup>th</sup>, 2025

## Table of Contents

|                                  |    |
|----------------------------------|----|
| Introduction.....                | 1  |
| Phase 1 .....                    | 2  |
| Phase 2 .....                    | 2  |
| Phase 3 .....                    | 3  |
| Preliminary.....                 | 3  |
| Rock-Paper-Scissors Example..... | 4  |
| Core Stability .....             | 7  |
| Analysis.....                    | 8  |
| Program Requirements .....       | 8  |
| Phase 1 .....                    | 8  |
| Phase 2 .....                    | 9  |
| Phase 3 .....                    | 10 |
| Program Walkthrough.....         | 10 |
| Overview .....                   | 10 |
| Primary Scenario .....           | 10 |
| Secondary Scenarios .....        | 11 |
| Functions Overview .....         | 12 |
| Experiments .....                | 15 |
| Methods .....                    | 15 |
| Results .....                    | 15 |
| Challenges.....                  | 15 |
| Conclusion .....                 | 16 |

## Introduction

When faced with a decision problem in which there is no choice that is objectively better than the others, it can be ambiguous as to which option is the best choice in each circumstance. Take for example, the hiring process for a vacant job position. While some candidates may not meet all requirements or may lack certain skills, it's likely that more than one candidate *will* be a very strong contender, with each candidate exceeding in different categories. In such case, there is no obvious decision to make solely based on the quantitative characteristics of each candidate. To inform our decision, we can analyze the relationships between candidates by using a *Tiered Coalition Formation Game*, a model introduced by Cory Siler in 2017. Tiered Coalition Formation Games are based on the idea of a *tier list*, where the choices available to choose from are grouped into *tiers*, with the most competitive choices at the top of the tier list, and the least competitive choices at the bottom. Once we have all candidates sorted into tiers, we can easily tell who the best performing candidates are, and therefore the best choices to make.

When determining which tier to put each candidate in, we need to consider the *utility* of each candidate. Utility may be thought of as a measure of each candidate's competitiveness among its peers. Higher utility indicates better overall performance, and lower utility indicates worse overall performance. Each candidate's goal is to maximize its utility by being placed in a tier with similarly competitive candidates. If a poor performing candidate is placed in the highest tier, it will be severely disadvantaged against other candidates in the tier, which is not desirable despite enjoying a higher placement. Generally, a *stable* tier list is one in which all agents are satisfied with their placement.

This independent study is designed with the goal of performing experiments on Tiered Coalition Formation Games to determine the time required to reach stability and the properties induced by certain types of stability. By the end of the study, we will have produced original source code for agent behavior simulation and testing environments for several types of stability, as well as a written report outlining key concepts explored and an interpretation of the testing results.

This independent study is broken down into 3 phases:

- Phase 1: Simulated Agent Behavior
- Phase 2: Probabilistic Matchup-Oriented Preferences
- Phase 3: Core Stability Experiments

## Phase 1

Phase one began with studying and discussing literature related to Tiered Coalition Formation Games including Siler, C. (2017) Tiered Coalition Formation Games, Arnold, N., Goldsmith, J., Snider, S. (2022) Extensions to Tiered Coalition Formation Games, and Arnold, N. (2025) Tiered Coalition Formation Game Variants, Stability, and Simulation.

After gaining a foundational understanding of Tiered Coalition Formation Games, we then shifted focus to simulating agent behavior in K-Tiered Coalition Formation Games. By implementing pseudocode outlined in the discussed literature and adding additional requirements that will be discussed in detail in the Analysis section, we produced a Java program that is able to determine and perform agents' preferred moves, resulting in a socially consciously stable tier list.

## Phase 2

Phase two introduced probabilistic preferences for agents. Under probabilistic matchup-oriented preferences, introduced by Nathan Arnold in Arnold, N., Goldsmith, J., Snider, S. (2022) Extensions to Tiered Coalition Formation Games, agents paired against each other have a percentage chance to win the matchup, and therefore, the outcome of a particular matchup is not always the same. For example, agent 1 may have a 30% (or 0.30) chance to win against agent 2, so, theoretically, 30% of the time agent 1 will win the matchup and 70% of the time agent 2 will win the matchup. In the Win matrix, preferences are calculated with the formula  $Win(i, j) = 2(p - 0.5)$ , where  $p$  is the probability for a win. This means that if an agent has a preference value against another agent that is greater than 0, then there is a probability greater than 50% that the agent will win the matchup. Likewise, if the preference value is less than 0, the probability for a win is less than 50%.

This preference model increases the applicability and real-world similarity of tiered coalition formation games by allowing matchups that are not only intransitive, but also stochastic. A common application of such problems is in game theory for character selection. Two given characters in a game may not be strictly better than one another, but one character may have certain advantages or abilities that make it a more viable choice in certain scenarios, thus giving a probabilistic preference.

### Phase 3

Phase 3 introduced the notion of core stability, which is the main topic of the study and will be discussed in detail later in this report. [Of course, need a lot more here]

## Preliminary

Tiered Coalition Formation Games (TCFGs) are a way to represent intransitive relationships among agents in a decision problem. Agents form *coalitions* (sets) with other similarly useful agents based on *utility* and *fitness*. These coalitions are sorted into strictly ordered *tiers* that categorize their usefulness against other agents in the tier list. This study deals specifically with K-Tiered Coalition Formation Games (KTCFGs), which are simply TCFGs that are restricted to exactly  $k$  tiers, where  $k \in \mathbb{N}$ .

The focus of the study is stability on K-Tiered Coalition Formation Games. For cohesion, we will provide definitions for the three main types of stability and related concepts.

- A *Nash stable* tier list is a tier list,  $T$ , such that there does not exist any agent,  $a_i$ , that can find a more-preferred tier list by moving from its current tier.
- A *blocking coalition* is a nonempty subset of agents,  $C$ , that could form a new tier in a tier list,  $T$ , such that each agent in  $C$  prefers their new position in the resulting tier list,  $T'$ .
- A *core stable* tier list is a tier list for which no blocking coalition exists.
- A *socially consciously stable* tier list is a tier list,  $T$ , in which there exists no agent,  $a_i$ , that can improve its utility by moving to a new position in the tier list without decreasing the overall utility of all other agents.

We can illustrate in detail how each of these components work by using the game Rock-Paper-Scissors as an example.

## Rock-Paper-Scissors Example

### Identifying Agents

In this game, there are three choices that a player may choose from: rock, paper, or scissors. In the context of TCFGs, these choices are called *agents* and can be represented by an index value. Rock will be agent 0, paper will be agent 1, and scissors will be agent 2.

This gives us the set  $a = \{0, 1, 2\}$ .

We initially pair all agents into one coalition before either manually or organically forming new coalitions to be sorted in the tier list. A coalition that contains all agents in the tier list is called the *grand coalition*.

### Developing a Win Matrix for Deterministic Matchup-Oriented Preferences

Now that we have identified our agents, we need a way to determine which agent is preferable in any given scenario. For this purpose, we will develop a *Deterministic Win Matrix* using *matchup-oriented preferences* based on the rules of the game. Rock always wins against scissors, but always loses against paper, paper always wins against rock, but always loses against scissors, and scissors always wins against paper, but always loses against rock. Any agent paired against itself always ends in a tie. We can represent a winning outcome as 1, a losing outcome as -1, and a tied outcome as 0. Since we are using deterministic matchup-oriented preferences, we will strictly use integers to represent preferences in the Win Matrix because the outcome of each matchup is predetermined. If any agent,  $a_i$ , is paired against any other agent,  $a_j$ , the outcome will always be the same regardless of how many times this pairing occurs. This gives us the following matrix:

| Win Matrix      | Agent 0 (Rock) | Agent 1 (Paper) | Agent 2 (Scissors) |
|-----------------|----------------|-----------------|--------------------|
| Agent 0 (Rock)  | 0              | -1              | 1                  |
| Agent 1 (Paper) | 1              | 0               | -1                 |

|                    |    |   |   |
|--------------------|----|---|---|
| Agent 2 (Scissors) | -1 | 1 | 0 |
|--------------------|----|---|---|

With this Win Matrix, we can find the *preference* of any agent by using the agents' indices. For example, if we want to find the preference of agent 0 (rock) against agent 2 (scissors), we can find  $Win(0, 2) = 1$ . This means that agent 0 wins against agent 2. We can find the overall *utility* of any agent by summing each preference of the agent. For example, agent 1 has a utility of  $Win(1, 0) + Win(1, 1) + Win(1, 2) = 0$ . In this example, all agents have a utility of 0. This means that there is no more-preferred agent in the set. In other words, no agent is necessarily better than any other agent overall.

### Determining Nash Stability

If we linearly assign each agent to a new tier in a tier list  $T$ , we can see that:

$$T = [[a_0], [a_1], [a_2]]$$

For clarity, we may also represent this as a table:

| Tier | Agent |
|------|-------|
| 0    | 0     |
| 1    | 1     |
| 2    | 2     |

For any agent,  $a_i$ , in the tier list, we say that  $a_i$  "sees" all agents in the same tier as  $a_i$  or any lower tier. This is denoted by  $Seen(a_i, T)$ , where  $a_i$  is any agent in the tier list,  $T$ .

Referencing the table for this example, we can determine that  $Seen(a_0, T) = \{a_0\}$ ,

$Seen(a_1, T) = \{a_0, a_1\}$ , and  $Seen(a_2, T) = \{a_0, a_1, a_2\}$ .

Using the Win Matrix, we can determine the utility of the tier list  $U(T)$  by using the following formulas:

$$U(T) = \sum_{i=0}^k u_i(T)$$

$$u_i(T) = \sum_{j \in \text{Seen}(a, T)} \text{Win}[a, j]$$

Below is a step-by-step walkthrough of how we can apply these formulas to find the overall utility of the tier list in this example:

$$u_0 = \text{Win}[0, 0]$$

$$= 0$$

$$u_1 = \text{Win}[1, 0] + \text{Win}[1, 1]$$

$$= 1 + 0$$

$$= 1$$

$$u_2 = \text{Win}[2, 0] + \text{Win}[2, 1] + \text{Win}[2, 2]$$

$$= -1 + 1 + 0$$

$$= 0$$

$$U(T) = u_0 + u_1 + u_2$$

$$= 0 + 1 + 0$$

$$= 1$$

Because  $U(T) \neq 0$ , there must be a preferable move in the tier list, and thus  $T$  is not Nash stable. Since  $u_1 \neq 0$ , we know that  $a_1$  must see a preferable move because  $a_1$  is the only agent in  $T_1$ . Let's consider what happens if  $a_1$  moves to  $T_2$ , so that we get the resulting tier list,  $T' = [[a_0], [a_1, a_2]]$ .



$$u_0 = \text{Win}[0,0]$$

$$= 0$$

$$u_1 = \text{Win}[1,0] + \text{Win}[1,1] + \text{Win}[1,2] + \text{Win}[2,0] + \text{Win}[2,1] + \text{Win}[2,2]$$

$$= 1 + 0 + (-1) + (-1) + 1 + 0$$

$$= 0$$

$$U(T) = u_0 + u_1$$

$$= 0 + 0$$

$$= 0$$

Because  $U(T) = 0$ , there are no preferable moves in the tier list, and thus  $T'$  is Nash stable. It was proven by Cory Siler in 2017 that if a tier list is Nash stable under matchup-oriented preferences, it is also core stable, thus  $T'$  is also core stable.

## Core Stability

The focus of this empirical study is core stability. We conjecture that if a  $k$ -tier list is not core stable, then there must exist a move by one or more agents that improves the overall utility of the tier list. If this is true, then by corollary, any tier list has the maximum possible utility for a given instance and number of tiers is core stable. Our experiments aim to provide data that will help in the formation of a proof of the conjecture.

Core stability is important as it is a prevalent concept in cooperative game theory. If we have a tier list that is not stable, such as a randomly generated tier list, there is no useful information that we can conclude from the tier list. In other words, it is useless. However, if we organize the agents in such a way that they start to enjoy their placements in the tier list, we are

then able to conclude the utility of an agent against its peers by looking at its tier. This is particularly useful for topics such as character selection.

As necessary as core stability is in game theory, it also has other real-world applications, such as the Gale-Shapley stable matching problem. In this problem, we have two sets, one of hospitals, and one of medical school students applying for residency. Assuming each student applies to all hospitals in the set, they are likely to have an order of preference for which hospital they would most like to spend their residency in. Likewise, each hospital has an order of preferences among the students for which they most want to work for them. For this problem, strict ordering is assumed. Once all students have applied, it is up to the hospitals to decide who they will extend an offer to. The first hospital will extend an offer to its most-preferred student. The receiving student will then decide to tentatively accept, or decline based on where the offering hospital lies in the student's preferences and whether the student has been made a more-preferred offer. This process repeats until all students have accepted positions. Once the process is complete, the hospitals will have each accepted their most-preferred student without a student working in more than one hospital, thus, the result is core stable. It is true that in each such problem, the proposing party will always have its most-preferred pairs, while the same may or may not be true for the recipients.

## Analysis

In this section, we will discuss how and why we have developed a Java program to simulate agent behavior in K-Tiered Coalition Formation Games.

## Program Requirements

### Phase 1

The goal of phase 1 is to have a foundational program that takes a set of agents,  $a$ , assigned to  $k$  tiers in a tier list,  $T$ , and a Deterministic Win Matrix for all agents in  $a$ , as input and produces a socially consciously stable tier list,  $T'$ , as console output. The functional

requirements for the final version of the program at completion of phase 1 are described as follows:

1. The system shall allow the user to input a Win Matrix from CSV file input.
2. The system shall allow the user to define a tier list through keyboard input.
3. The system shall be capable of generating a random Win Matrix with deterministic preferences for  $n$  agents,  $n \in \mathbb{N}$ .
4. The system shall be capable of generating a random tier list based on user-given values for  $n$  and  $k$ ;  $n, k \in \mathbb{N}$ .
5. The system shall produce a socially consciously stable tier list for any valid user-given or randomly generated tier list.
6. The system shall allow the user to input or generate as many tier lists as they wish, one at a time, displaying results after each input or generation.
7. The system shall have a clear, concise, and intuitive text-based menu.
8. The system shall check all user input and only accept valid data.
9. The system shall display all output in a descriptive, comprehensible manner.

With this program, we are able to run large-scale test cases using deterministic matchup-oriented preferences to determine the time required for any given tier list to reach socially conscious stability. This program is the framework to our experimentation environment and may be easily modified to accommodate diverse types of K-Tiered Coalition Formation Games, such as those with probabilistic matchup-oriented preferences.

## Phase 2

Programmatically, phase 2 was relatively simple. While the Win matrix, utility list, and fitness list dealt with integers in phase one for deterministic preferences, we simply needed to accommodate floating point values for these elements to handle probabilistic preferences.

## Phase 3

In phase 3, we implemented functions in a new class, `CoreStabilityTest`, to take a Win matrix, tier list, number of agents, and number of tiers as input, and produce a Boolean value indicating whether the tier list is core stable. With this testing code, we are able to generate random instances of Tiered Coalition Formation Games and determine whether they are core stable or not. Combined with what we have created in previous tiers, we may keep track of the number of test cases ran and output all data for test cases that meet a certain criterion (iterations to reach stability greater than  $n^2$  in our case).

Once we were able to determine if a tier list is core stable, we implemented the experimentation functions. [\[Talk more about this\]](#)

## Program Walkthrough

In this section, we will give a high-level walkthrough of the Java program.

### Overview

The program consists of several interconnected functions that work together to provide three main functionalities to the user.

1. Create a tier list either randomly or from user input.
2. Determine the number of iterations taken to reach socially conscious stability.
3. Output the resulting socially consciously stable tier list and the number of iterations taken as console output, and optionally, as CSV file output.

### Primary Scenario

[\[This needs to be changed. It should be random probabilistic preferences with core stable test. I need to wait until my program is fully finished to iron out the exact details of how this will work, so it is left as is for now.\]](#)

The most expected scenario is executing a test case based on CSV file input and outputting the results of the test case as CSV file output. We will outline how the program works in the primary scenario.

1. Upon program execution, the main menu is displayed.
2. The user shall enter “4” to select option four, test case based on CSV file input.
3. The user shall enter the exact file path to the CSV file they wish to import.
4. The system shall parse the CSV file for a Win Matrix.
5. The system shall calculate the number of agents based on the length of the Win Matrix.
6. The user shall enter the number of tiers required in the tier list.
7. The system shall output the Win Matrix to the console.
8. The system shall output the grand coalition to the console.
9. The user shall enter integers, separated by commas, representing the agent(s) to add to each tier. This will be repeated from  $0 \dots k - 1$ .
10. The system shall output the specified tier list to the console.
11. The system shall perform a local search on the tier list.
12. The system shall output the resulting socially consciously stable tier list and the number of iterations taken to the console.
13. The users shall enter ‘Y’ or ‘y’, indicating a request to save the result to a CSV file.
14. The user shall enter a name for the file to be saved.
15. The system shall output the resulting socially consciously stable tier list and the number of iterations taken to a CSV file named with the user-given title.
16. The system shall display a save confirmation message to the console.
17. The system shall display the main menu.
18. The user shall repeat the processes as many times as they wish.
19. The user shall enter “5” to select option five, Exit.
20. The system shall display a farewell message to the console.
21. The system shall terminate.

## Secondary Scenarios

There are many possible valid and invalid secondary scenarios that may occur. Valid secondary scenarios include:

1. Executing all hard-coded test cases.
2. Executing a randomly generated deterministic test case and saving the results.

3. Executing a randomly generated deterministic test case and not saving the results.
4. Executing a randomly generated probabilistic test case and saving the results.
5. Executing a randomly generated probabilistic test case and not saving the results.
6. Executing a test case via user input and not saving the results.
7. Immediately terminating the program.

Any invalid secondary scenarios, such as invalid user input, are handled by the program to ensure that no invalid or incorrect results are displayed and that the program does not prematurely terminate. Valid secondary scenarios follow a similar flow to the primary scenario. Because of this, as well as the amount of valid secondary scenarios, we will not outline them in this document.

## Functions Overview

[I will need to add all functions added since the last update to this report. I will wait until the program is complete to avoid having to use time redoing this with each draft]

In this section, we will alphabetically list each function and briefly describe its purpose and how it relates to other functions in the system.

### **bestPermittedMove()**

- Finds the most preferred move that a specified agent can make.
- Called by *kTierLocalSearch()*.

### **createRandomTierList()**

- Generates a random tier list based on passed values for the number of tiers and the set of agents.
- Called by *runRandomTestCase()*.

### **createTierList()**

- Generates a tier list based on passed values for the number of tiers and the set of agent, forming coalitions based on user keyboard input.

- Called by *runTestCaseFromFile()*.

**generateRandomDeterministicWinMatrix()**

- Generates a random Win Matrix with deterministic preferences based on a passed value for the number of agents.
- Called by *runRandomTestCase()*.

**generateRandomProbabilisticWinMatrix()**

- Generates a random Win Matrix with probabilistic preferences based on a passed value for the number of agents.
- Called by *runRandomTestCase()*.

**kTierLocalSearch()**

- Produces a socially consciously stable tier list derived from a passed tier list and returns the number of iterations taken to reach stability.
- Called by *runRandomTestCase()* and *runTestCaseFromFile()*.

**provideFilePath()**

- Gets a valid CSV file path from user keyboard input.
- Called by *runTestCaseFromFile()*.

**provideK()**

- Gets a valid integer value for the number of tiers for a tier list from user keyboard input.
- Called by *runTestCaseFromFile()*.

**provideN()**

- Gets a valid integer value for the number of agents from user keyboard input.
- Called by *runTestCaseFromFile()*.

**readWinMatrixFromFile()**

- Parses a Win Matrix from a CSV file.
- Called by *runTestCaseFromFile()*.

**runAllTestCases()**

- Executes and compares expected vs. actual results for every hard-coded test case.
- Called by *main()*.

**runRandomTestCase()**

- Executes a randomly generated test case.
- Called by *main()*.

**runTestCaseFromFile()**

- Executes a test case based on user input.
- Called by *main()*.

**saveResultToCSV()**

- Outputs the results of a test case into a CSV file.
- Called by *runRandomTestCase()* and *runTestCaseFromFile()*.



## Experiments

### Methods

TBD

### Results

TBD

## Challenges

This independent study offered many challenges that have increased my confidence in programming ability, technical reading comprehension and writing skills, time management, and oral presentation.

The first and most exacerbating challenge was the initial overwhelming nature of the task. Starting this study already a few weeks late into the semester, it was very difficult for me to find any sort of momentum initially as expanding upon scholarly articles is something I have never even thought about how to do before. With direction from Dr. Arnold and after reaching the first few milestones, it became much easier to know what to do next, how to do it, and when I needed to ask for help.

Initially, understanding the concepts and how they work from a low-level perspective was very confusing and difficult. This only made the issue of time worse because I didn't know how to go about understanding what I was reading in the scholarly articles assigned. Eventually, going through it slowly and recreating my own examples of the topics discussed solidified my knowledge and understanding, as well as discussing the topics in a less formal manner with Dr. Arnold.

There were almost too many times to count during this study that my task for the week was to hunt down small logical errors in my code that were entirely invalidating the results. I feel

confident in my ability to implement pseudocode into working Java code, but often there would be at least one small mistake that took me days to track down. I feel that, while this was a very frustrating aspect of the study, it greatly increased my understanding of the concept, the code, and my ability to debug my own code.

Time was a very big stressor in this independent study. From getting a late start, the time taken to gain momentum, errors in my code, and the sometimes-overwhelming amount of work needed from week to week, it took a lot of effort and time management to be able to complete the code project, the experiments, the oral presentation, and this written report by the end of the semester. Ultimately, I feel that, while very stressful, it gave me good experience in time management and task prioritization that were helpful in not only this study, but also the other courses that I was enrolled in during this semester.

## Conclusion

We ran some experiments and found some cool knowledges that will help us do some other, related super-duper cool thingies... with the stuffs that we found in them experiments. 👍